

Some preliminary notes and feedback for the “Abelian-square densities under bounded local avoidance: a non-monotone ternary family” article.

The notation h for the longest half-interval under consideration is problematic in Combinatorics on Words research, since the letters h and g are typically used as symbols for morphisms. Perhaps the Greek lowercase eta or ksi would be a better choice in this field.

Many concepts are introduced without definition. Sure, they may be defined in other articles, but the reader typically cannot know that.

For example, the term primitive appears in nine places.

What does it mean in this context?

The term primitive has several meanings in Combinatorics on Words.

It usually means certain kinds of words or morphisms.

About section

2 Bounded-avoidance languages and window presentations

What does the bold 1 in (2) mean?

The bold 1 also appears in Lemma 6.1.

Is it a vector of 1s, or what?

Please clarify.

And please explain the new concepts throughout the article.

Also visualise the graphs introduced. This would be enormously helpful for the reader.

Another thought and a research topic suggestion

The 3-letter case in question is VERY challenging, and no substantial progress has been made on it for decades, even though it has been studied mathematically and computationally. The long words on the page

https://algebra.fi/keranen/research/web110117/aa2f15796_aa2f25379_aa2fr1928+More.html

are the result of very long computer runs and do not derive from desired first principles, at least according to current understanding.

First, a couple of definitions from the website above.

aa2f:

A word is called aa2f (almost abelian square-free) if it does not contain an abelian square uv as a factor - with the exception that the shortest squares of period 1 ($|u| = |v| = 1$) are allowed. In other words, the repetitions xx and xxx , where x is a letter, are allowed to appear but all the other abelian squares should be avoided.

aa2fr;

A word is called aa2fr if it does not contain pure repetitions of the form xx , i.e., factors of the form $yxxz$, where x, y, z are letters and $y \neq x, z \neq x$. Note that the letter r in aa2fr stands for this restriction.

I have a properly trimmed/pruned, but still very long list of aa2f (and aa2fr) words over three letters. The length of these words is 40. Anyhow, the corresponding adjacency (or contact) matrix would be huge (though sparse).

As a new type of idea, perhaps one could use massively parallel computing that would take advantage of the Watson-Crick complementary DNA structure, where one of the double strands would consist only of letters of the alphabet $\{a,b,c\}$, i.e., say, $\{A, T, C\}$ and the other, imagine the upper, strand, would consist of the letters in $\{a,b,d\}$, i.e., in $\{A, T, G\}$.

As we know, bonding happens by the pairwise attraction of bases: A bonds with T, and C bonds with G. Therefore, unordered pairs (A, T) and (C, G) are referred to as complementary pairs of bases.

Below, I will use only the letters of the alphabet $\{a,b,c,d\}$, or just $\{a,b,c\}$.

The situation could apparently be arranged so that the lower string would only contain letters from the alphabet $\{a,b,c\}$ and the upper string would only contain letters from the alphabet $\{a,b,d\}$ (so the letter d would always be present at the top in place of the c at the bottom, and a and b would alternate according to the complementarity principle).

Now let's prepare a solution with trillions of 40-character aa2f or aa2fr chunks, and arrange them (perhaps with CRISPR-Cas9 technology) in such a way that

the “lower and upper parts” connect to each other exactly halfway, like Lego blocks. The result would be all kinds of long strings, but also long aa2f or aa2fr words.

Also note that in the case of a 3-letter alphabet, all letters a,b,c; or a,b,d; must appear in aa2f words of length 11, and in the case of aa2fr, in words of length 10.

Nevertheless, isolating those (automatically and naturally) pre-computed long aa2f or aa2fr result words from the solution is challenging and, as far as I understand, remains unsolved, though long strings can be separated from shorter ones by standard chromatographic methods.

However, modern computer tools computationally connect all this to the original problem addressed in the article. Here we do not need to consider complementarity in the biological sense.

The situation would look roughly like this (when word boundaries are added):

```
| aaabacbcccaaacbbbacc . caaabaacaaabbbcbba | . . .
| aaabaacaaabbbbaacccaaabacbcccaaacbbbacc | caaabaacaaabbbcbbaabbbcccaaabaacabcbba | . . .
```

In this case, the resulting word of length 80 is indeed aa2fr.

However, preliminary experiments indicate that only a small fraction of the longer strings produced by this method avoid abelian squares (note: I tend to write abelian with a small a).

In any case, abelian squares of length less than or equal to 20 (half-length ≤ 10) are always avoided.

A natural question arises: How does the probability of obtaining a word free from almost abelian squares (aa2f or aa2fr) change as a function of the word length?